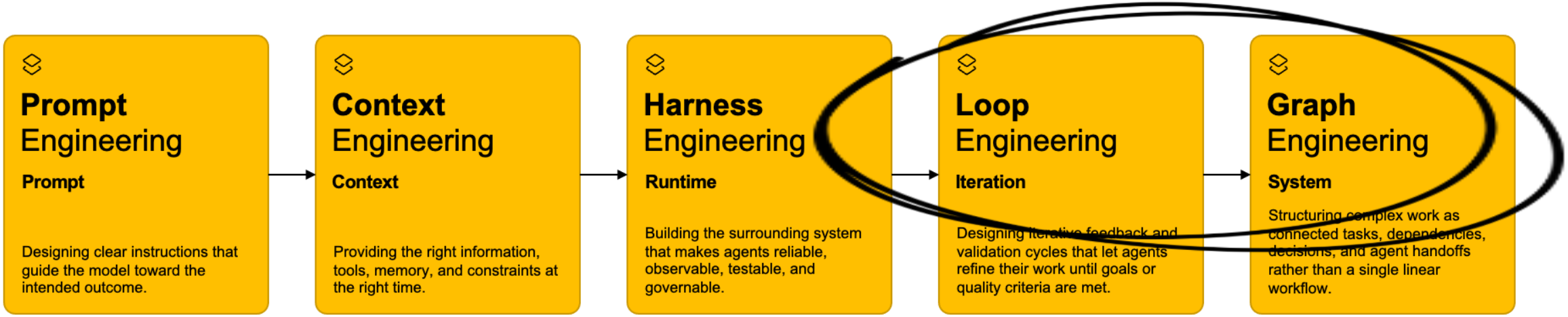


twoday

Loop and graph engineering



What should the agent achieve?

Fix the readiness check so whitespace-only values count as missing.

What does the agent need to know?

The readiness rules, relevant source files, and existing test expectations.

What is the agent equipped and allowed to do?

Read the repository, edit within the workspace, and run the .NET tests.

How should feedback determine the next action?

Reproduce the failure, repair it, rerun the same check, and stop or retry based on the result.

What routes can the work take, and what determines the route?

A passing check goes to review, a repairable failure goes back to the writer, and a blocker goes to the operator.

Loop engineering – using evaluation

Loop engineering is about designing how an agent uses feedback to decide whether to continue, repair, or stop.

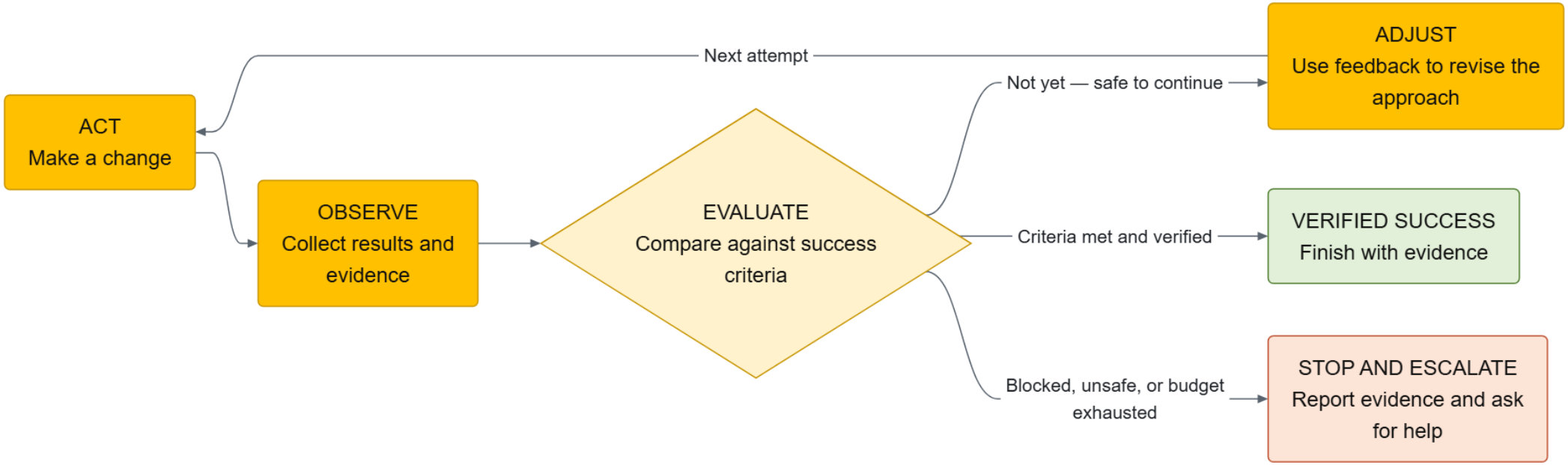
How should feedback determine the next action?

Reproduce the failure, repair it, rerun the same check, and stop or retry based on the result.

For example, the harness gives the agent a way to run tests.

Loop engineering determines when to run them, how failures influence the next change, and whether the evidence is sufficient to finish.

Loop engineering – using evaluation



Act → Observe → Evaluate → Adjust → Act again
With two exits: verified success or stop and escalate.

Loop engineering

demo

From a prompt to proof

Fix the checkout calculator to satisfy the documented pricing rules.

Run the existing tests, use failures to guide your changes, and rerun the relevant checks after each change.

Do not modify the acceptance tests or weaken the requirements.

Allow at most three repair attempts.

Stop earlier if you cannot make progress and explain the blocker.

Only claim completion when the required checks pass.

What happens
Implement the checkout calculation.
A test reports: expected **€95**, received **€90**.
The result violates the shipping rule; another attempt is allowed.
Revise the approach: determine free shipping **after** applying the
Apply the correction and collect fresh results.

Loop engineering – resources

We’re not engineering an agent that always gets the first answer right.
We’re engineering a process that can detect when it is wrong, correct itself, and stop responsibly.

Put the goal in the **issue**, the repeatable procedure in a **skill**, and ownership in a **custom agent**.

Issue template	Skill	Custom agent
Goal	**Repeatable procedure**	**Ownership**

Issue template

Done when
Saving a release note with a whitespace-only service name shows the required-field error.

How to check
Repeat the reported browser steps. Run the existing release validation tests.

Work limit
Allow up to two edit-and-check cycles. Stop as soon as both checks pass.
If the limit is reached, report the remaining failure and leave the diff for review.

Skill

run-bounded-loop/SKILL.md

1. Read the issue's goal, check, and work limit. Resolve any ambiguity that affects what counts as done.
2. Reproduce the failure or inspect existing evidence such as a failed CI check.
3. Make a scoped change and repeat the agreed check. Count that as one cycle.
4. Stop on a pass, a blocker, or the limit. Otherwise use the latest result to choose the next change.
5. Report the diff, observed results, cycles used, and remaining work.

Custom agent

issue_worker

name = "issue_worker"
description = "Makes one scoped change and verifies it against an assigned issue."
developer_instructions = Use the supplied goal, scope, and verification steps.
Inspect the latest evidence, make one scoped change, and run the agreed check.
Return the diff, observed result, and any blocker to the coordinator.

Loop engineering resources

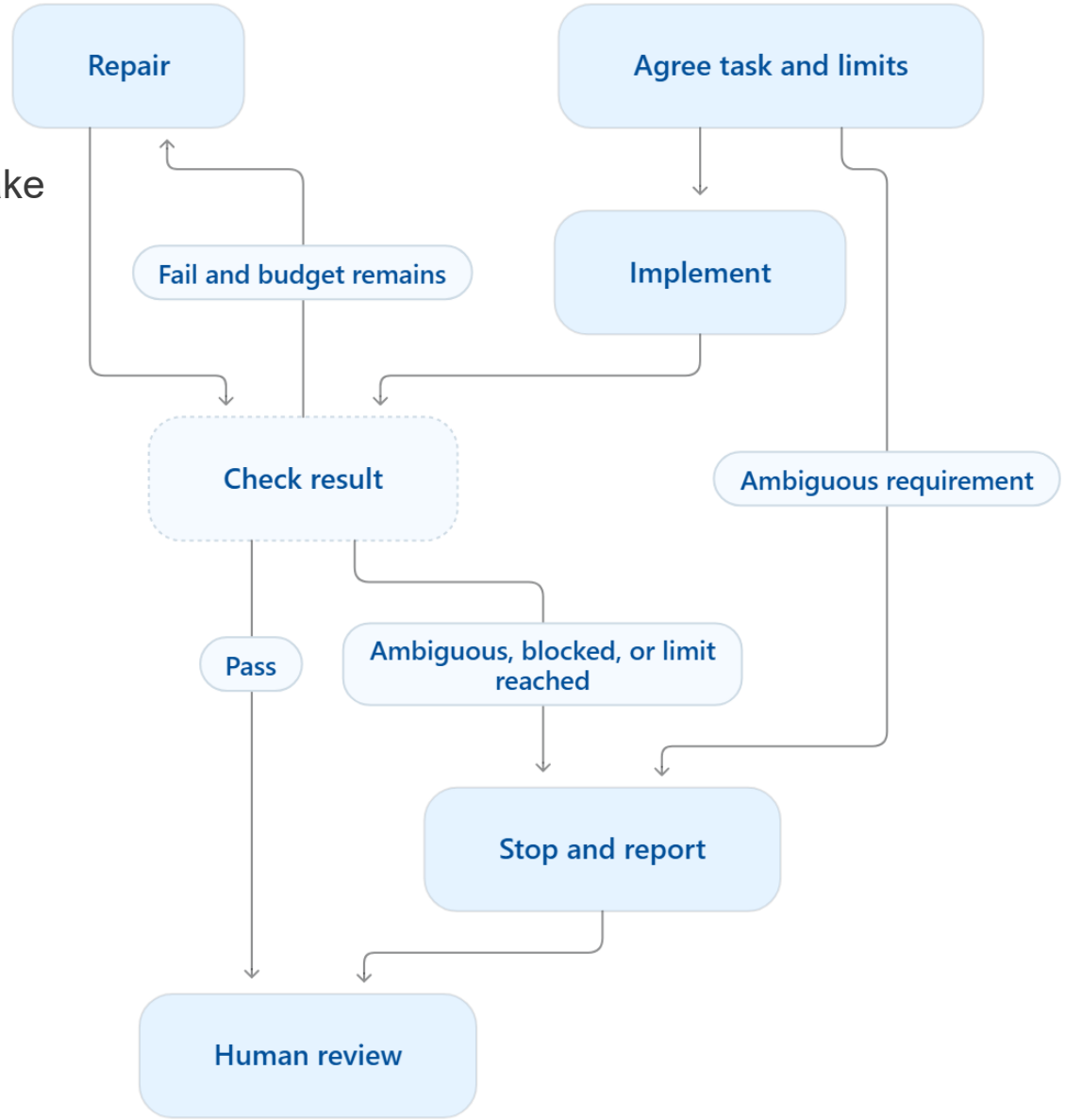
demo

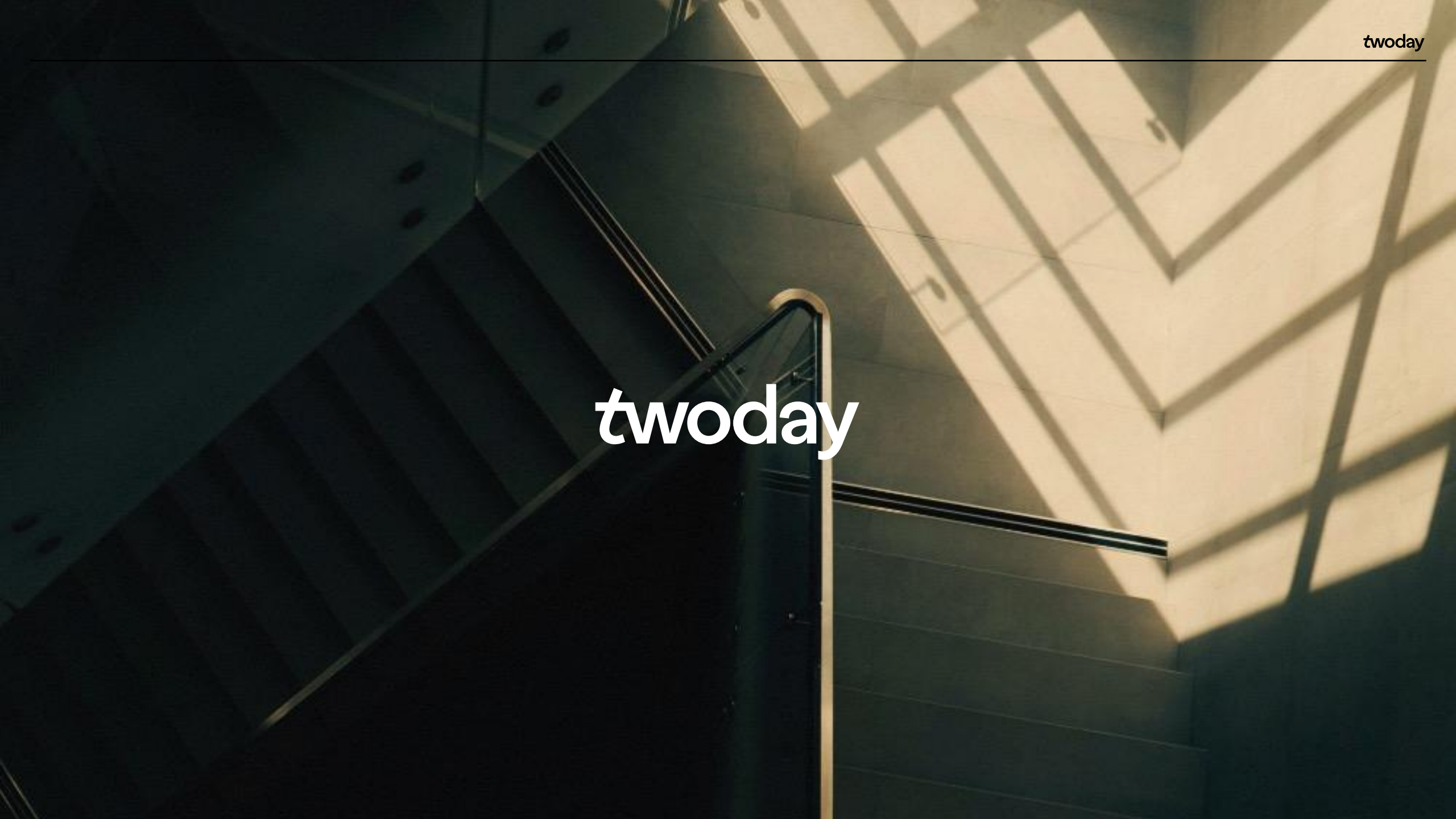
Graph engineering

Graph engineering is about designing the paths work can take and the conditions that route it between steps or people.

What routes can the work take, and what determines the route?

A passing check goes to review, a repairable failure goes back to the writer, and a blocker goes to the operator.




twoday